

Week-4 Practical

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2, child3;
    int status;

    printf("Parent PID: %d\n", getpid());

    child1 = fork();

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 finished\n");
        exit(10);
    }

    child2 = fork();

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 finished\n");
        exit(20);
    }

    child3 = fork();

    if (child3 == 0) {
        printf("Child 3: PID = %d\n", getpid());
        sleep(2);
        printf("Child 3 finished\n");
        exit(30);
    }

    // wait() waits for any one child
    wait(&status);

    if (WIFEXITED(status)) {
        printf("Parent: One child finished with exit status %d\n",
               WEXITSTATUS(status));
    }

    // waitpid() waits for a specific child
    waitpid(child1, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child 1 finished with exit status %d\n",
               WEXITSTATUS(status));
    }

    waitpid(child3, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child 3 finished with exit status %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent: All children completed.\n");

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ gcc src/multiple_children.c -o
bin/multiple_children
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ls bin
builtin_shell      path_resolver      waitpid_demo
multiple_children  variable_expand
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ./bin/multiple_children
Parent PID: 695
Child 1: PID = 696
Child 2: PID = 697
Child 3: PID = 698
Child 2 finished
Parent: One child finished with exit status 20
Child 3 finished
Child 1 finished
Parent: Child 1 finished with exit status 10
Parent: Child 3 finished with exit status 30
Parent: All children completed.
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ |
```

2. Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

PART A — Create a Zombie

```
GNU nano 8.7.1 src/zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting now...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent will NOT call wait().\n");
        printf("Parent sleeping for 20 seconds...\n");

        sleep(20);

        printf("Parent finished.\n");
    }

    return 0;
}
```

Output in Terminal-1:

```
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ gcc src/zombie.c -o bin/zombie
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ./bin/zombie
Parent Process
Parent PID: 773
Child Process
Child PID: 774
Parent will NOT call wait().
Parent sleeping for 20 seconds...
Child PID: 774
Child is exiting now...
Parent finished.
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$
```

Commands executed in Terminal-2 to find Zombie:

```
priyanka@DESKTOP-HOISO7A:~$ cd ~/OS_Skill/Week-4
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ps -el
| grep zombie
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$
```

PART B — Remove the Zombie Using `wait()`

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");

        waitpid(pid, &status, 0);

        printf("Child has been collected.\n");

        if (WIFEXITED(status))
        {
            printf("Child exited normally.\n");
            printf("Exit status: %d\n", WEXITSTATUS(status));
        }

        printf("No zombie process remains.\n");
    }

    return 0;
}
```

Output:

```
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ nano src/zombie_fixed.c
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ gcc src/zombie_fixed.c -o bin/zombie_f
ixed
priyanka@DESKTOP-HOISO7A:~/OS_Skill/Week-4$ ./bin/zombie_fixed
Parent Process
Parent PID: 936
Child PID: 937
Parent is waiting for child...
Child Process
Child PID: 937
Child is exiting...
Child has been collected.
Child exited normally.
Exit status: 0
No zombie process remains.
```